

Implementación técnica

ECOMMIFY

Etapla 2 — Implementación técnica completa en PostgreSQL y MongoDB

Unidad 6: Proyecto final

MATERIA	ENTREGABLE
Diseño y Optimización de Bases de Datos	Etapla 2 — Evaluativa Junio 2026

INTEGRANTES DEL GRUPO	
1	Juan José Vélez Álvarez
2	Juan Sebastián Buitrago Romero
3	Sergio Andrés Cobos Suárez
4	David Aníbal Vásquez Beltrán

Índice

a. Executive Summary	3
b. Introducción y contexto	4
c. Arquitectura y diseño.....	6
d. Implementación técnica	10
e. Evaluación de rendimiento y escalabilidad	14
f. Análisis arquitectónico crítico	14
g. Recomendaciones estratégicas	15
h. Conclusiones	16

a. Executive Summary

El presente informe consolida los resultados y decisiones arquitectónicas del proyecto Ecommify, una plataforma de comercio electrónico diseñada para gestionar el ciclo de vida completo de compras en línea (basada en el ecosistema Olist). El sistema modela nueve áreas principales: clientes, pedidos, productos, pagos, reseñas, vendedores, geolocalización, traducciones e ítems de pedido.

Se implementó una Arquitectura Híbrida (Políglota) que combina un núcleo transaccional en PostgreSQL (Supabase) y un repositorio documental en MongoDB (Atlas). Esta estrategia fue validada resolviendo anomalías de actualización presentes en el análisis exploratorio (EDA), donde la estructura original (orders_full) violaba la 1FN mediante grupos repetitivos.

Principales logros y métricas de éxito:

M1 (Tiempos de respuesta): Optimización en consultas transaccionales y analíticas mediante el uso estratégico de índices.

M2 (Almacenamiento): Consolidación del despliegue en MongoDB Atlas M0, garantizando un almacenamiento menor a 512 MB.

M3 (Integridad): Estabilización del modelo relacional mediante la deduplicación del catálogo de geolocalización.

M4 (Consistencia): Diseño de una estrategia de sincronización (por lotes en fase actual) para reconciliar product_id.

Recomendaciones estratégicas clave: Escalar hacia una sincronización orientada a eventos (CDC con Debezium y Kafka) para mitigar el replication lag, y migrar a clústeres MongoDB M10+ en configuraciones Multi-AZ para habilitar Change Streams y estrategias de Sharding reales en producción.

b. Introducción y contexto

Problema de negocio

Ecommify es una plataforma de comercio electrónico basada en el dataset Brazilian E-Commerce de Olist, que integra información de clientes, pedidos, productos, pagos, vendedores, reseñas y geolocalización. El principal reto identificado fue gestionar simultáneamente cargas transaccionales altamente estructuradas y cargas analíticas con información heterogénea y semiestructurada. Esto exigió diseñar una arquitectura capaz de garantizar integridad transaccional, flexibilidad documental y escalabilidad.

Los hallazgos del EDA evidenciaron:

- Más de 100.000 pedidos.
- Más de 99.000 reseñas.
- 32.951 productos.
- 73 categorías de productos.
- Más de un millón de registros geográficos.

Además, se identificó alta consistencia relacional entre entidades transaccionales y una fuerte heterogeneidad en el catálogo de productos y comentarios de usuarios.

Objetivos y alcance

Objetivo general

Diseñar e implementar una arquitectura híbrida PostgreSQL + MongoDB que soporte las necesidades operativas y analíticas de una plataforma de comercio electrónico.

Alcance

El proyecto abarca:

- Diseño conceptual y lógico.
- Implementación física en PostgreSQL (Supabase).
- Implementación documental en MongoDB Atlas.
- Optimización mediante índices, particionamiento y aggregation pipelines.
- Evaluación cuantitativa de rendimiento.
- Diseño teórico de alta disponibilidad mediante Replica Sets y Sharding.

- Análisis arquitectónico basado en el Teorema CAP.

Metodología aplicada

La metodología utilizada fue incremental y basada en evidencia:

Fase 1 – Exploración de datos (EDA)

- Identificación de entidades.
- Calidad de datos.
- Cardinalidades.
- Patrones de consulta.
- Volumen esperado.

Fase 2 – Diseño conceptual

- Modelo Entidad-Relación.
- Restricciones de negocio.
- Identificación de entidades principales.

Fase 3 – Diseño lógico

- Normalización a 3FN en PostgreSQL.
- Diseño documental en MongoDB.
- Selección de patrones de modelado.

Fase 4 – Implementación física

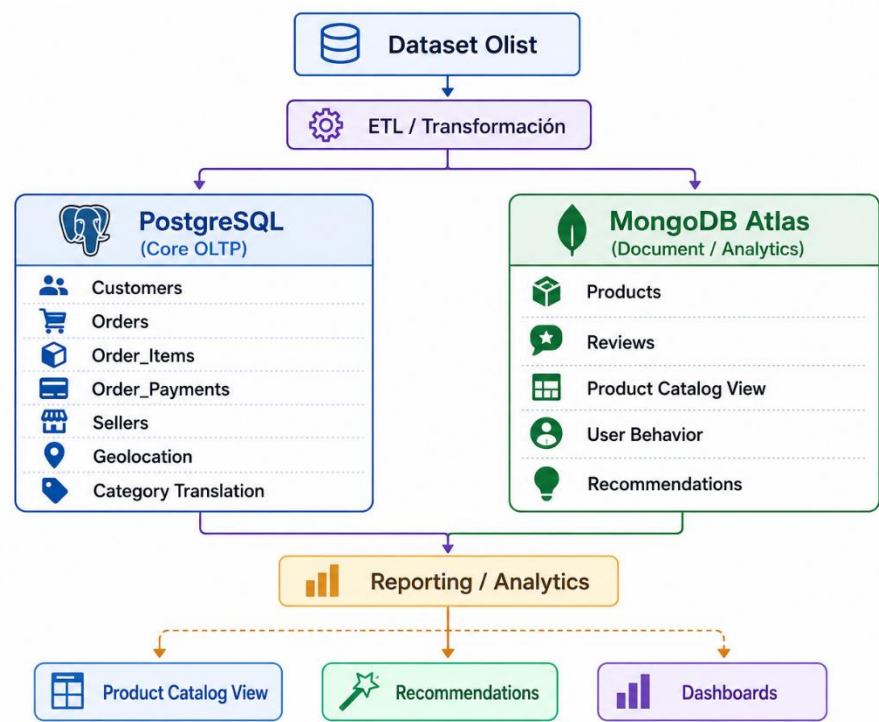
- Creación de esquemas.
- Índices.
- Validadores.
- Aggregation Pipelines.

Fase 5 – Optimización

- EXPLAIN ANALYZE.
- executionStats.
- Particionamiento.
- Índices compuestos ESR.
- Simulación de Sharding y Replica Sets.

c. Arquitectura y diseño

Diagrama de arquitectura híbrida completa



Matriz de decisiones, PostgreSQL vs MongoDB

MATRIZ DE DECISIONES – SELECCIÓN DE MOTOR DE DATOS

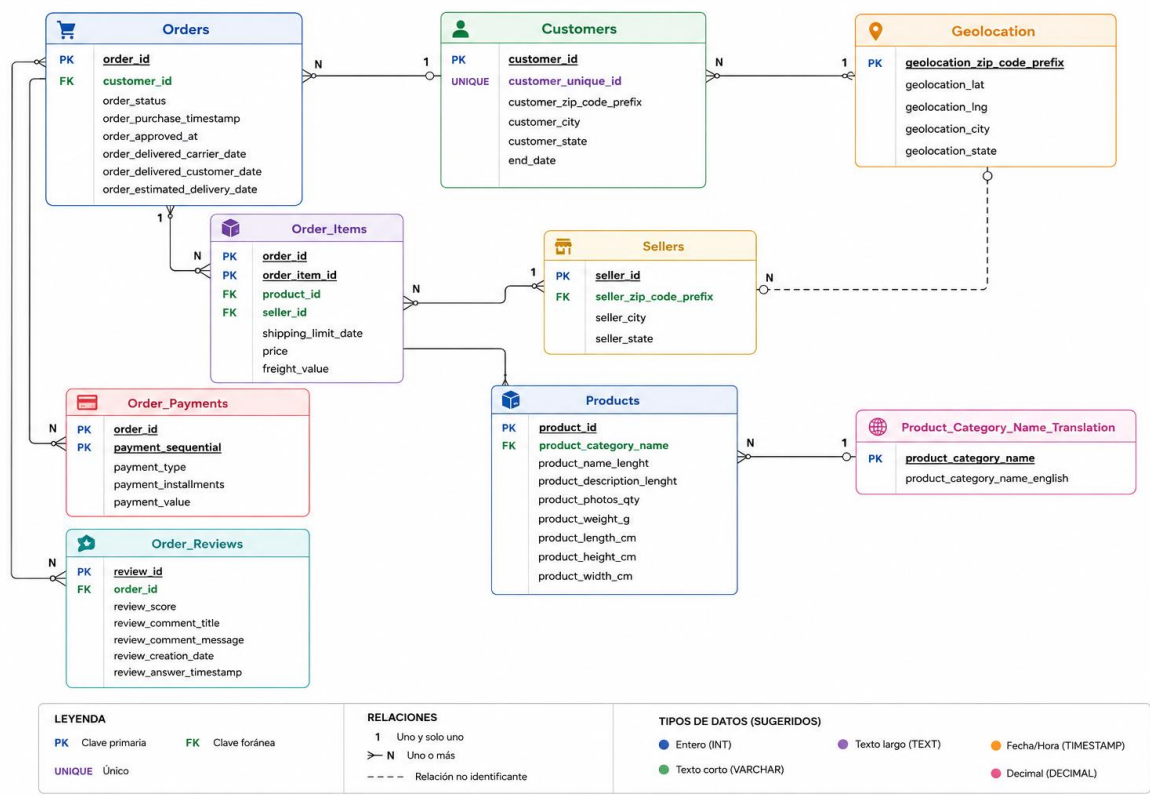
Entidad	Motor	Justificación
Customers	PostgreSQL	Integridad referencial
Orders	PostgreSQL	Transacciones ACID
Order_Items	PostgreSQL	Relaciones complejas
Order_Payments	PostgreSQL	Consistencia financiera
Sellers	PostgreSQL	Trazabilidad
Geolocation	PostgreSQL	Datos estructurados
Category Translation	PostgreSQL	Catálogo maestro
Products	MongoDB	Atributos heterogéneos
Reviews	MongoDB	Texto libre
Product Catalog View	MongoDB	Consultas optimizadas
User Behavior	MongoDB	Eventos variables
Recommendations	MongoDB	Datos precalculados

Análisis del teorema CAP aplicado por Módulo

 Módulo	 Prioridad CAP	 Justificación
 Orders	CP Consistencia + Partición	 Consistencia obligatoria. Las órdenes requieren integridad total en todas las operaciones.
 Payments	CP Consistencia + Partición	 No se toleran inconsistencias. Los pagos deben procesarse con exactitud y no pueden perderse.
 Customers	CP Consistencia + Partición	 Integridad relacional. La información de clientes se relaciona con múltiples entidades críticas.
 Products	AP Disponibilidad + Partición	 Alta disponibilidad. El catálogo de productos debe estar siempre disponible para los usuarios.
 Reviews	AP Disponibilidad + Partición	 Consistencia eventual aceptable. Las reseñas pueden propagarse de forma eventual sin afectar la operación.
 Recommendations	AP Disponibilidad + Partición	 Datos analíticos. Las recomendaciones se basan en datos analíticos y pueden ser recalculadas.
 Product Catalog View	AP Disponibilidad + Partición	 Optimización de lectura. Vista optimizada para consultas rápidas y experiencia de usuario eficiente.

Modelos ER y Documentales (Sintéticos)

- Modelo entidad relación



- Modelo documental MongoDB

MODELO DOCUMENTAL – MONGODB ATLAS

Products

```
{
  _id: ObjectId("..."), // product_id
  sku: "PROD_12345",
  name: "Smartphone XYZ",
  category: {
    id: "electronics.smartphones", // categoria del producto
    name: "Smartphones",
    name_en: "Smartphones"
  },
  specs: {
    brand: "BrandX",
    model: "X12",
    color: "black",
    storage: "128GB",
    RAM: "8GB",
    dimensions: {
      height_cm: 15.0,
      width_cm: 7.2,
      depth_cm: 0.8
    },
    weight_g: 180,
    warranty_months: 12
  },
  analytics: {
    views: 12543, // métricas agregadas
    purchases: 842,
    conversion_rate: 0.067,
    avg_rating: 4.3,
    review_count: 256
  },
  created_at: ISODate("2023-01-01T00:00:00Z"),
  updated_at: ISODate("2024-05-01T00:00:00Z")
}
```

Reviews

```
{
  _id: ObjectId("..."), // review_id
  product_id: ObjectId("..."), // referencia a Products
  customer_id: ObjectId("..."),
  created_at: ISODate("2024-04-15T10:30:00Z"),

  score: 5, // calificación
  comment: {
    es: "Excelente producto, cumple con todo lo esperado. Muy recomendable."
  },
  analysis: {
    sentiment: "positive",
    keywords: ["excelente", "cumple", "recomendable"],
    language: "es",
    spam: false,
    translated: {
      en: "Excellent product, it meets all expectations. Highly recommended."
    }
  }
}
```

Product Catalog View

```
{
  _id: ObjectId("..."), // vista agregada por producto
  product_id: ObjectId("..."),
  product_name: "Smartphone XYZ",

  metrics: {
    total_reviews: 256,
    avg_rating: 4.3,
    total_purchases: 842,
    conversion_rate: 0.067,
    trend_30d: "up"
  },

  top_sellers: [
    { seller_id: ObjectId("..."), seller_name: "Seller A", sales: 120, avg_price: 899.90 },
    { seller_id: ObjectId("..."), seller_name: "Seller B", sales: 98, avg_price: 909.00 }
  ],

  category: {
    id: "electronics.smartphones",
    name: "Smartphones",
    path: ["electronics", "devices", "smartphones"]
  },
  updated_at: ISODate("2024-05-01T00:00:00Z")
}
```

User Behavior

```
{
  _id: ObjectId("..."), // evento de comportamiento
  customer_id: ObjectId("..."), // referencia a Customers
  device: {
    type: "mobile",
    os: "Android",
    browser: "Chrome",
    version: "124.0.0.0"
  },
  events: [
    { event: "view_item", product_id: ObjectId("..."), timestamp: ISODate("2024-05-01T10:15:00Z"), metadata: { position: 1, source: "home" } },
    { event: "add_to_cart", product_id: ObjectId("..."), timestamp: ISODate("2024-05-01T10:16:30Z"), quantity: { quantity: 1 } },
    { event: "purchase", order_id: ObjectId("..."), timestamp: ISODate("2024-05-01T10:20:45Z"), metadata: { payment_method: "credit_card", value: 899.90 } }
  ]
}
```

Recommendations

```
{
  _id: ObjectId("..."), // recomendación generada
  customer_id: ObjectId("..."), // referencia a Customers
  generated_at: ISODate("2024-05-01T11:00:00Z"),
  model_version: "v2.1.0",

  recommendations: [
    { product_id: ObjectId("..."), score: 0.95, reason: "based_on_purchase_history", position: 1 },
    { product_id: ObjectId("..."), score: 0.89, reason: "similar_users", position: 2 },
    { product_id: ObjectId("..."), score: 0.82, reason: "category_affinity", position: 3 }
  ]
}
```

d. Implementación técnica

Resumen de implementación PostgreSQL

- **Tipos avanzados utilizados**

Tipo		Uso
	JSONB	Atributos variables de productos
	Arrays	URLs y etiquetas
	Composite Types	Dimensiones físicas
	tstzrange	Ventanas de entrega

- **Extensiones**

- pgcrypto
- pg_trgm
- btree_gin
- btree_gist
- postgis
- pg_stat_statements

- **Índices principales**

idx_orders_status_ts
idx_order_items_order
idx_reviews_negativas
brin_orders_purchase_ts
gin_products_tags

gin_orders_metadata
gist_geolocation

- **Particionamiento**

- **Orders**
PARTITION BY RANGE(order_purchase_timestamp)
- **Geolocation**
PARTITION BY LIST(geolocation_state)

Resumen de implementación MongoDB

- **Patrones de modelado aplicados**

Patrón		Aplicación
	Embedding	Categorías, dimensiones, eventos
	Referencing	product_id, order_id
	Computed Pattern	Métricas agregadas
	Subset Pattern	Product Catalog View
	Extended Reference Pattern	Recomendaciones

- **Índices implementados**

Products

{ product_id: 1 }

{ category.name_en: 1 }

```
{
  category.name_en: 1,
  analytics.avg_review_score: -1,
  analytics.total_sales: -1
}
```

Reviews

```
{ review_id: 1 }
{ order_id: 1 }
{ score: 1 }
```

TEXT

```
{
  "comment.title": "text",
  "comment.message": "text"
}
```

- **Aggregation Pipelines**
 - Catálogo por categoría.
 - Análisis de reseñas.
 - Productos más vendidos.

Todos con más de cinco stages y optimizados mediante índices ESR y allowDiskUse.

Decisiones técnicas NO obvias y justificación

1. Mantener Products en ambos motores

Decisión

- PostgreSQL almacena versión normalizada.
- MongoDB almacena catálogo enriquecido.

Justificación

Permite mantener integridad transaccional mientras se optimizan búsquedas y recomendaciones.

2. Shard Key compuesta

```
{  
  "category.name_en": 1,  
  "_id": "hashed"  
}
```

Justificación

Una shard key únicamente por categoría generaría hotspots debido a la concentración de productos en categorías populares. El hash sobre `_id` distribuye uniformemente los documentos manteniendo eficiencia de consulta.

3. Deduplicación de Geolocation antes de la carga

Problema

Más de un millón de registros geográficos excedían restricciones del tier gratuito de Supabase.

Solución

ETL previo en Colab para eliminar duplicados.

Resultado

Reducción aproximada de:

1.000.000+ registros

↓

~150.000 registros únicos

Esto permitió implementar restricciones e índices sin comprometer almacenamiento ni rendimiento.

e. Evaluación de rendimiento y escalabilidad

Análisis Comparativo PostgreSQL vs MongoDB

Aspecto Evaluado	Ganador	Justificación
Integridad Estructural	PostgreSQL	Control estricto mediante FKs y restricciones de dominio. Garantiza que no existan ítems sin un pedido válido.
Flexibilidad de Esquema	MongoDB	Permite la evolución del catálogo de productos sin costosas migraciones ALTER TABLE.
Velocidad de Lectura (Catálogo)	MongoDB	Al evitar JOINS complejos, la lectura del producto y su categoría embebida requiere un solo executionTimeMillis mínimo.

Identificación de Cuellos de Botella: El principal limitante encontrado no fue el volumen de datos (estabilizado gracias al modelado lógico), sino las restricciones de infraestructura del *Free Tier*: incapacidad de habilitar verdaderas réplicas configurables y ausencia de eventos CDC (Change Data Capture) nativos.

f. Análisis arquitectónico crítico

Trade-offs y Consistencia Eventual

Al dividir el sistema, Ecommify introdujo Consistencia Eventual. Esto crea ventanas de inconsistencia o Replication Lag:

Fricción Operativa (Read-Your-Own-Writes): Si un cliente realiza un pago e inmediatamente es redirigido a ver su orden, podría impactar un nodo secundario retrasado, viendo "Pago Pendiente". Mitigación: Implementar Causal Consistency Sessions a nivel del Driver.

Overselling (Reserva de Inventario): Vender stock inexistente por leer un secundario obsoleto. Mitigación: Aplicar readConcern: "majority" y forzar una validación síncrona en PostgreSQL justo antes del checkout final.

Reflexión de Selección de Tecnología

La selección fue correcta. PostgreSQL es innegociable para proteger el núcleo del negocio. Sin embargo, la arquitectura políglota agrega una carga cognitiva y operativa alta. Para un MVP, un monolito sobre PostgreSQL (usando campos JSONB para productos) podría haber bastado,

pero para la proyección a escala de Ecommify, la separación prepara el terreno para microservicios independientes.

g. Recomendaciones estratégicas

Plan de Escalamiento (10x Crecimiento) y Migración a Producción

Infraestructura: Migrar a MongoDB Atlas M10+ y bases de datos RDS/Cloud SQL dedicadas. Esto

habilita el paso de sincronización batch a Real-Time CDC utilizando Apache Kafka y Debezium (arquitectura orientada a eventos).

Replica Sets: Configurar 3 nodos en Multi-AZ (1 Primary, 2 Secondaries) para garantizar alta disponibilidad y procesos de elección seguros.

Sharding: Implementar Sharding en MongoDB. Se requiere una estrategia de Compound Shard Key que balancee la distribución uniforme de los datos y la eficiencia de las consultas dirigidas, previniendo los "jumbo chunks".

Tecnologías Complementarias

Se recomienda la introducción de una capa de Caching (Redis) para sesiones y tokens, Elasticsearch para búsquedas avanzadas difusas de productos (full-text search superior a las regex de MongoDB), y herramientas de Observabilidad (Datadog/New Relic) para monitorear el lag de replicación y la salud del ecosistema híbrido.

h. Conclusiones

El proyecto Ecommify culmina demostrando que la optimización del rendimiento no ocurre en el vacío; cada partición e índice fue justificado por patrones de acceso reales descubiertos en la Etapa 1. El equipo logró conciliar la rigidez transaccional con la flexibilidad documental, cumpliendo con las normativas de la Maestría.

La reflexión final revela que las limitaciones tecnológicas de los entornos gratuitos operaron como catalizadores de creatividad arquitectónica, forzando decisiones como la sincronización por lotes controlada y el diseño idempotente, habilidades fundamentales en el diseño de software a gran escala.